

第九章基于昇腾的智能小车实践

9.1 目标

智能小车 AI 应用的实现需要从底层硬件执行、模型训练和推理决策等功能的协同实现。在前面不同章节，我们已经完成了 Linux 操作、Python 基础及进阶知识的学习，实现了对边缘开发板的了解和初步使用，学习了 ESP32 控制板的使用、串口通信实现及 ESP32 硬件主程序设计。接下来我们要进一步实现小车的组装了解基于边缘开发板的智能小车开源代码，理解小车控制的实现逻辑、AI 推理应用实现以及开发者套件与 ESP32 进行指令通信的接口程序。

本章介绍如何使用麦克纳姆轮、单目相机、电机以及其驱动板、路由器、Atlas 200I DK A2 开发板和配套耗材组装出一辆小车，并控制小车实现自主识别地标与障碍并做出相应反应、自动巡线的任务。图 9.1 车道线示意图给出了车道线图示，本章任务要求小车能够实现如下功能。

- 1) 小车从停车点 P 外侧出发，以地标指定方向，分别绕内外赛道各走一圈，最终返回停车点（P 点所在的红色区域内）。
- 2) 在识别到转向、掉头、停车等各类地标的时候，小车能够做出对应的反应。
- 3) 在识别出行人的时候，小车能够主动停止直到行人离开后继续。
- 4) 保持在两条车道线内（不限制颜色）进行运动，不能越线。

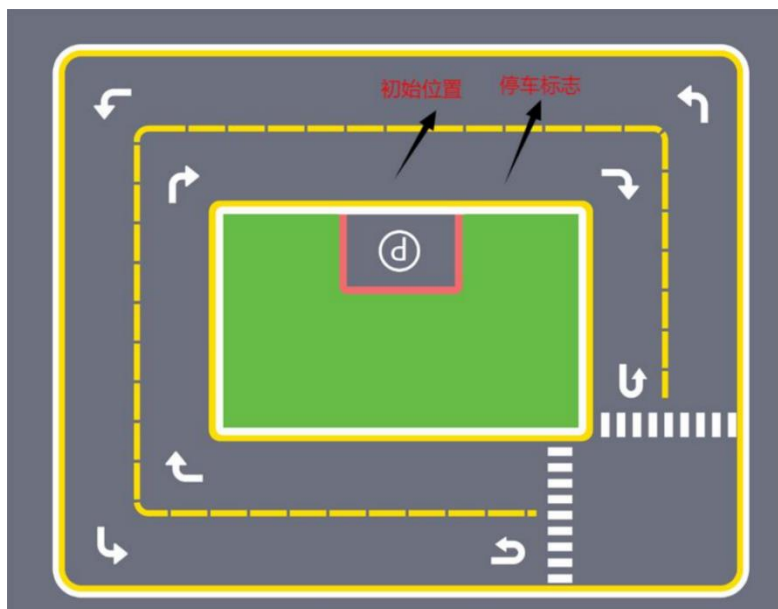


图 9.1 车道线示意图

9.2 小车装配

图 9.2 小车底盘部件图示和成车图示图给出了小车底盘的各个部件和安装完成后的图示。正式安装前，结合调试过程中的线缆拔插、充电等各项需求，需要规划下小车活动部件的放置位置和线缆走线。我们按照从下至上的顺序组装各连接件。大致顺序为电池> TT 直流电机 > 麦克 纳姆轮 > 广角摄像头模块 > ESP32 单片机 > Atlas 200I DK A2 开发者套件顺序组装组装小车，四个轮子需要拧螺丝。对于各种线缆，规划好走线后需要进行必要的固定，提升可靠性和美观度。



(a) 底盘示意图



(b) 成车图示

图 9.2 小车底盘部件和成车图示

9.2.1 电机接线

普通的 TT 电机需要将排线的一头剪开，分别焊接到电机上的焊点中。如下图 9.3 电机焊接完成效果图所示。带编码的 TT 电机，将一端 6p 的 XH2.54 公头插入电机母座，另外一端，则按照各根线的不同信号连接至对应的位置。下面给出了 GMR 编码形式的 TT 电机的端口各线的定义。其中电机线+/电机线-和 ESP32 开发板上的四个电机驱动端口相接；编码器电源和编码器地线对应的杜邦母线头接入开发板上的 5V 端口和地线。编码器 A 相和 B 相需合理规划开发板上的 GPIO 端口并进行分配后接入。开源代码里没有电机的编码驱动方法，需要各组在直驱的基础上自行实现。

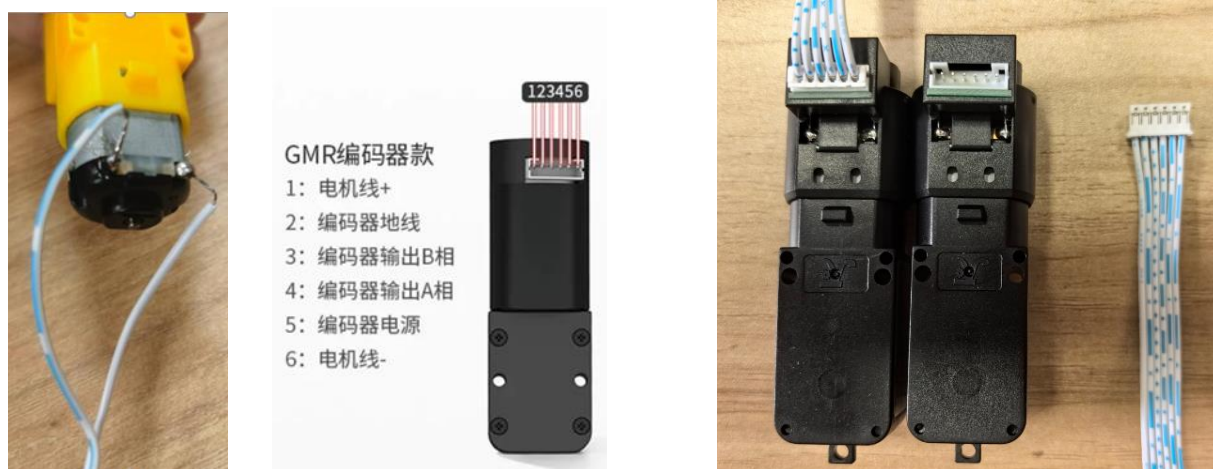


图 9.3 电机连接参考图

9.2.2. 底盘组装

小车的机械组件包含麦轮、电机。

1 电机组装

如下图 9.4 所示，将电机固定在支撑板上。

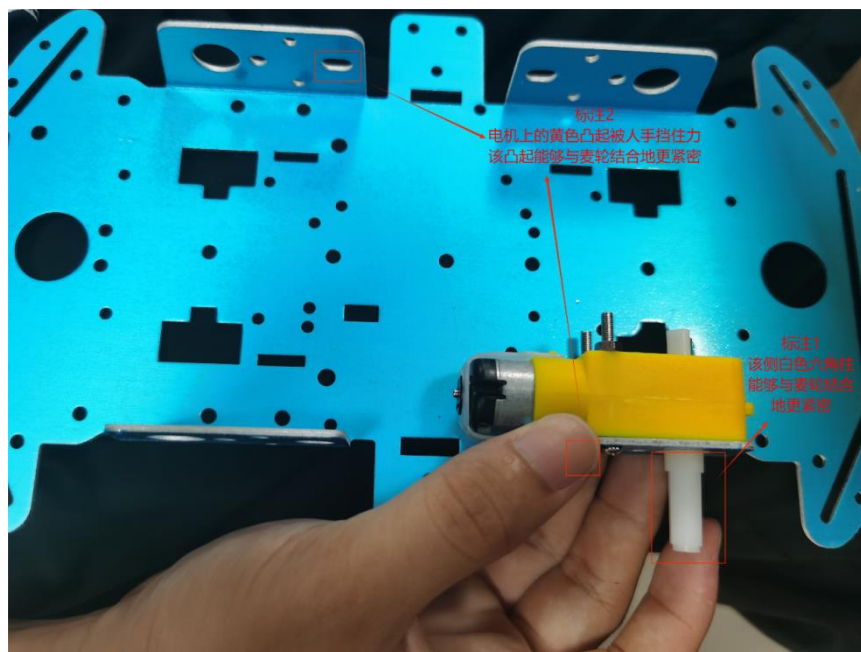


图 9.4 电机固定

2 麦轮组装

麦轮运动方式如下图 9.5 X 型麦轮运动分解图所示，注意各个轮子的轮辊的方向要如图所示。

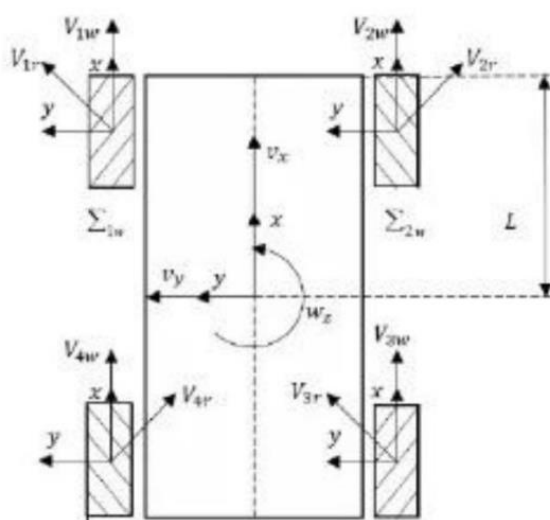


图 9.5 X 型麦轮运动分解

麦轮组装分为 X 型和 O 型(此处采用 X 形)。通过运动学向量合成可以求出，每个轮子的不同运动方向和速度组合会导致整个底盘的平移、自转、差速运动，可以实现机器人在二维平面上的全向移动。每个轮子的旋转方向和速度的组合决定了地盘的移动方向和速度。其具体控制方式如下。

前进：所有轮子均正转（标记为“++++”）。

后退：所有轮子均反转（标记为“----”）。

左平移：前左轮和后左轮反转（“--”），前右轮和后右轮正转（“++”）。

右平移：前右轮和后右轮反转（“--”）前左轮和后左轮正转（“++”）。

左前方 45 度：前左轮正转（“+”），后左轮反转（“-”），前右轮和后右轮静止（“00”）。

右前方 45 度：前右轮反转（“-”），后右轮正转（“+”），前左轮和后左轮静止（“00”）。

逆时针旋转：前左轮和后左轮反转（“--”），前右轮和后右轮正转（“++”）。

顺时针旋转：前左轮和后左轮正转（“++”），前右轮和后右轮反转（“--”）。

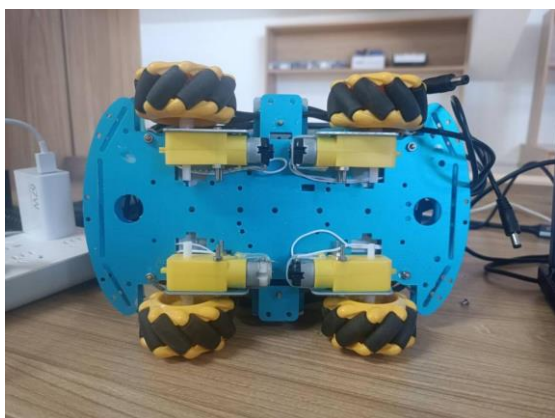


图 9.6 底盘组装示例图

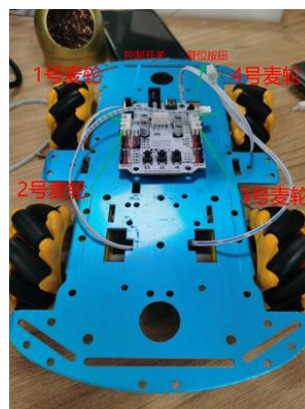


图 9.7 电机驱动板接线图

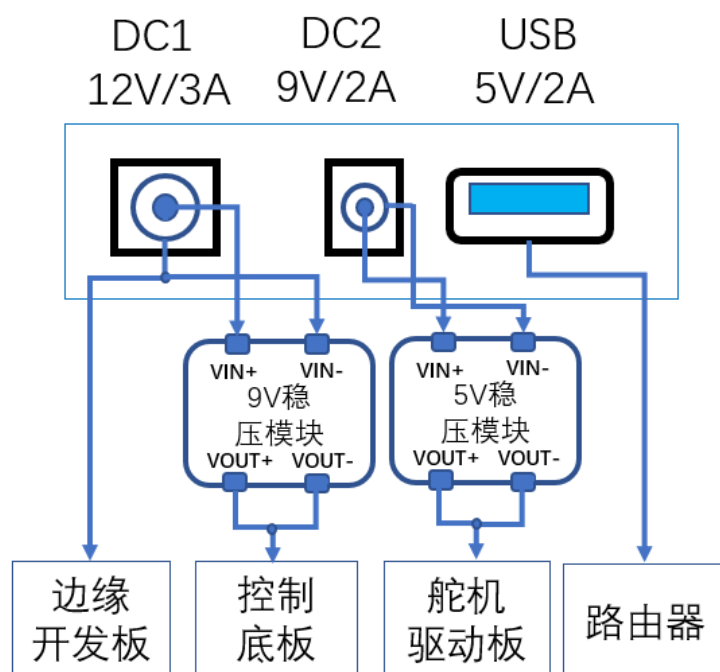
电机和麦轮组装好后，图 9.6 底盘组装示例所示。图 9.7 电机驱动板接线图给出了电机驱动板安装图例。以 X 型麦轮为例，将 1 号麦轮的排线插入电机驱动板的 12、13 号位，2 号麦轮的排线插入 14、15 号位，以此类推。控制开关存在两种状态：一侧为开启，拨至该侧后可以控制电机旋转；另一侧为关闭，拨至该侧后，无论上位机发送何种信息，电机都不旋转。复位按钮在电机存在无法停止/转速过慢/没有转动等异常状态时按下，将电机状态重置。

9.2.3 供电和数据线连接

小车的电气和信号连接主要是供电网络和信号连接，考虑到小车有 2-3 层，且各个部件没有固定的放置位置，从方便走线、方便日常拆卸检修和小车配重的角度，需要在合理规划小车部件放置的基础上根据下面的供电和信号拓扑进行合理的布线。

1 供电

稳定的电源供给，是小车正常运行的基本保证，本次组装为小车提供的是 12000mAh（以上）容量的多输出锂电池，可以为小车系统提供 12V/3A、9V/2A、5V/2A 的输出。当电池容量下降的时候，输出电压也会下降，为了提升小车驱动的稳定性和稳定性，我们为小车的电机驱动板提供了由 MP1584EN 稳压芯片构成的稳压模块进行供电。小车整车接线拓扑图如下图 xxx 小车接线拓扑图所示。其中 DC1 同时接到边缘开发板和 9V 稳压模块，需要使用一分二的电源端子进行输出，并在其中的一个端子焊接上稳压模块。图 xxxx 稳压模块连接图给出了具体的连接方式。我们在焊接的时候，一定要注意端子的红线和稳压模块的 VIN+ 焊接，端子的黑线和稳压模块的 VIN- 焊接，如果焊接错误，会导致稳压模块损坏。对于使用舵机的底盘，还需要通过 DC2 为舵机提供一路 5V 稳压输出。建议使用带编码的电机来改善电压跌落导致的运



行状态不稳定的情况。稳压模块为可选部件，本次不提供。

图 9.8 稳压模块连接图

2 数据线连接

控制底板通过其 Ttype-C 端通过数据线连接在 Atlas 200I DK A2 的口上。USB 摄像头连接在开发板的 USB Type-A 口上，此处 USB 口的分配不作要求。为避免出现数据传输问题，需要将摄像头连接在开发板的 USB 口而非通过 USB 拓展坞转接。边缘开发板通过 eth0 口和路由器的 LAN 口相连。控制底板的四个电机驱动端口通过带端子的连接线和

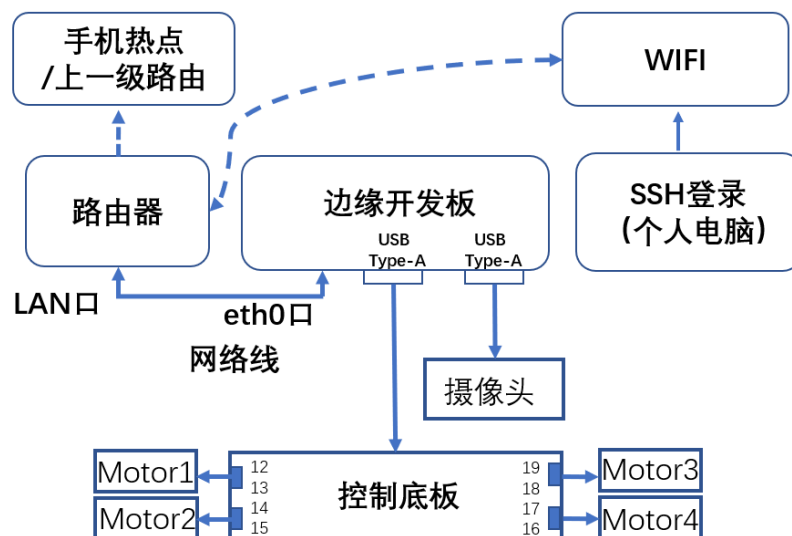


图 9.9 小车接线拓扑图

9.3 智能小车 demo 测试

在第 7 章相关部分，我们已经完成了了 `arduino` 环境安装，`esp32` 开发板的连接，并完成了通过编译、下载源代码实现了 `esp32` 开发板的运行，我们还通过自己编写代码和配置电气连接，实现了对 `sg90` 舵机的转动控制，在本节，我们需要带智能小车的底盘控制代码进行解读和测试。与智能小车 `demo` 实现相关的车道线检测模型、地标检测模型的生成过程在 9.4 节进行说明。

9.3.1 电机驱动理解

前面第 8 章，我们引导读者打开本书随附的开发板代码 `ascend-devkit-master\src\E2E-Sample\Car\ESP32` 下的 4 个文件“`ESP32_Servo.cpp`”、“`ESP32_Servo.h`”、“`RL_ESP32_Motor.cpp`”、“`RL_ESP32_Motor.h`”复制到 `Arduino` 库文件路径下，此处为：
`C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\hardware\esp32\2.0.17\cores\esp32` 不同用户对应的用户名和版本号会不同。现在，我们来分析下这些代码的构成。

1. `ESP32_Servo.h`

这个文件是舵机库头文件，其定义了 `Servo` 类、舵机控制相关常量和函数声明，给 `.cpp` 文件提供了接口规范。

MIN_PULSE_WIDTH 定义舵机可识别最短高电平脉冲，单位为 us，500us 对应 0°，2500us 对应 180°。该值可以根据不同型号舵机机的参数进行调整。DEFAULT_PULSE_WIDTH 给出了舵机连接时候的默认脉宽（角度，90°），实践中可能需要微调这些值确保角度正确。

REFRESH_CPS 定义了 PWM 频率为 50Hz，REFRESH_USEC 定义其周期为 20ms（20000us），DEFAULT_PULSE_WIDTH_TICKS 定义了舵机对应 20000us 周期中，高电平脉冲 2000us 的脉冲计数 4825ticks，1500us 对应 3619ticks，500us 对应 1206ticks；它们分别对应 180、90、0 的 pwm 脉冲高电平时长对应的 ticks，不需要调整。

该文件中，Servo 类声明了公有方法 attach()、detach()、write()和 read() 等（Arduino 标准舵机接口），私有方法角度、微秒和 tick 转换计算，成员变量通道号、引脚、最小/最大脉冲和 PWM 参数。

2. ESP32_Servo.cpp

该文件是舵机库的实现文件，其实现 Servo 类的所有功能，把角度指令转为 ESP32 硬件 PWM 信号后基于 ESP32 硬件 LEDC_PWM 生成舵机控制信号，其包含如下功能。

ESP32 有 16 路 LEDC 通道，库自动分配/回收通道。其中 ServoCount 统计已用舵机数量，ChannelUsed 标记通道占用状态（0 未用 / 1 使用 / -1 释放可重用）。attach() 函数调用 ledcSetup() 设置 50Hz PWM，调用 ledcAttachPin()将 PWM 通道绑定到 GPIO。write() 函数的输入 < 500 时当作角度（0~180°），其调用 map()把角度转为脉冲宽度（us），最终调用 writeMicroseconds()写脉冲。该函数把微秒转为硬件 LEDC 的计数值（tick），其中 ledcWrite() 输出 PWM 波形。sToTicks()函数实现了单位转换，其将微秒值转换为 PWM 计数值，ticksToUs()则将计数值转换为微秒值。代码调用 ESP32 HAL 库 esp32-hal-ledc.h（官方 LEDC PWM 驱动），实现驱动。

3. RL_ESP32_Motor.h

该文件是电机库头文件，其作用为定义 Motor 类，声明电机控制函数，依赖 ESP32 MCPWM 硬件驱动。其关键内容包含依赖 driver/mcpwm.h（ESP32 电机专用 MCPWM 控制器）和 Motor 类成员，其中有私有成员方向引脚、速度引脚、MCPWM 单元和定时器配置及公有成员构造函数、mcpwm_begin()和 Motor_Speed()。

4. RL_ESP32_Motor.cpp

该文件基于 ESP32 官方 MCPWM 驱动 driver/mcpwm.h 实现了电机驱动，其可以实现驱动直流电机正转、反转、调速、刹车。构造函数 Motor(num, PinA, PinB)根据电机编号（1~6）分配 MCPWM 单元/定时器，绑定方向引脚、速度引脚。mcpwm_begin() 初始化；mcpwm_gpio_init() 绑定引脚到 MCPWM，设置 PWM 频率为 1000Hz，初始化占空比为 0（电机停止）。Motor_Speed(duty_cycle) 调

速，正数为正转，A 输出低电平，B 输出 PWM；负数为反转，B 输出低电平，A 输出 PWM；0 为刹车，A/B 都输出高电平。

5. car_ctr_esp32.ino plaintext

car_ctr_esp32.ino 中包含了头文件 ESP32_Servo.h 和 RL_ESP32_Motor.h，并调用 ESP32_Servo.cpp、RL_ESP32_Motor.cpp、esp32-hal-ledc.h、driver/mcpwm.h 和 ESP32 硬件 PWM 资源。

该文件首先调用 Motor 类构造函数创建电机对象（4 个电机（编号，方向/速度引脚）并实例化：

```
Motor motors[] = {  
    Motor(1, 12, 13),  
    Motor(2, 14, 15),  
    Motor(3, 16, 17),  
    Motor(4, 18, 19)  
};
```

然后舵机对象（2 个舵机），Servo servo_25;Servo servo_26，随后 setup () 初始化硬件。接着调用库函数 mcpwm_begin()初始化 4 个电机的 MCPWM：

```
for (int i = 0; i < 4; i++) {  
    motors[i].mcpwm_begin();  
}
```

调用库函数 attach() 初始化 LEDC_PWM，绑定舵机引脚：

```
servo_25.attach(25, 500, 2500);  
servo_26.attach(26, 500, 2500);
```

接着使用 loop () 接收串口指令，控制马达和舵机硬件：

```
set_motor(speeds);  
set_servo(angles);
```

其中电机控制函数 set_motor ()调用库函数：Motor_Speed(速度)，使用该语句：

```
motors[i].Motor_Speed(speeds[i]);
```

实现速度控制，速度值正数正转、负数反转、0 刹车。

舵机控制函数 set_servo ()调用库函数 write(角度)，使用该语句：

```
servo_25.write(angles[0]);
```

实现 PWM 脉冲控制舵机转动

9.3.2 小车底盘实现

小车底盘的各个组成部分中，主控 **ESP32** 承担信号接收与发送，执行并调动其他部分完成指令；直流电机为小车提供动力；舵机则是执行器，装载摄像头，可控角度 $0\sim180^\circ$ ；超声波传感器实现 $2\sim400\text{cm}$ 的非接触距离感测功能，测距精度可达 3mm ，确保障碍检测；小车载板提供支架，电池提供电源。

1 麦克纳姆轮原理及方向控制

如下图 9.10 麦克纳姆轮示意图所示，我们只需调节 4 个电机的转动方向及速度即可实现小车的各种运动方式。图 9.11 麦克纳姆轮运动分解示意图给出了不同方向的运动分解图示。

麦克纳姆轮原理：

这种全方位移动方式是基于一个有许多位于机轮周边的轮轴的中心轮的原理上，这些成角度的周边轮轴把一部分的机轮转向力转化到一个机轮法向力上面。

依靠各自机轮的方向和速度，这些力的最终合成在任何要求的方向上产生一个合力矢量从而保证了这个平台在最终的合力矢量的方向上能自由地移动，而不改变机轮自身的方向。在它的轮缘上斜向分布着许多小滚子，故轮子可以横向滑移。小滚子的母线很特殊，当轮子绕着固定的轮心轴转动时，各个小滚子的包络线为圆柱面，所以该轮能够连续地向前滚动。麦克纳姆轮结构紧凑，运动灵活，是很成功的一种全方位轮。有 4 个这种新型轮子进行组合，可以更灵活方便的实现全方位移动功能。



图 9.10 麦克纳姆轮示意图

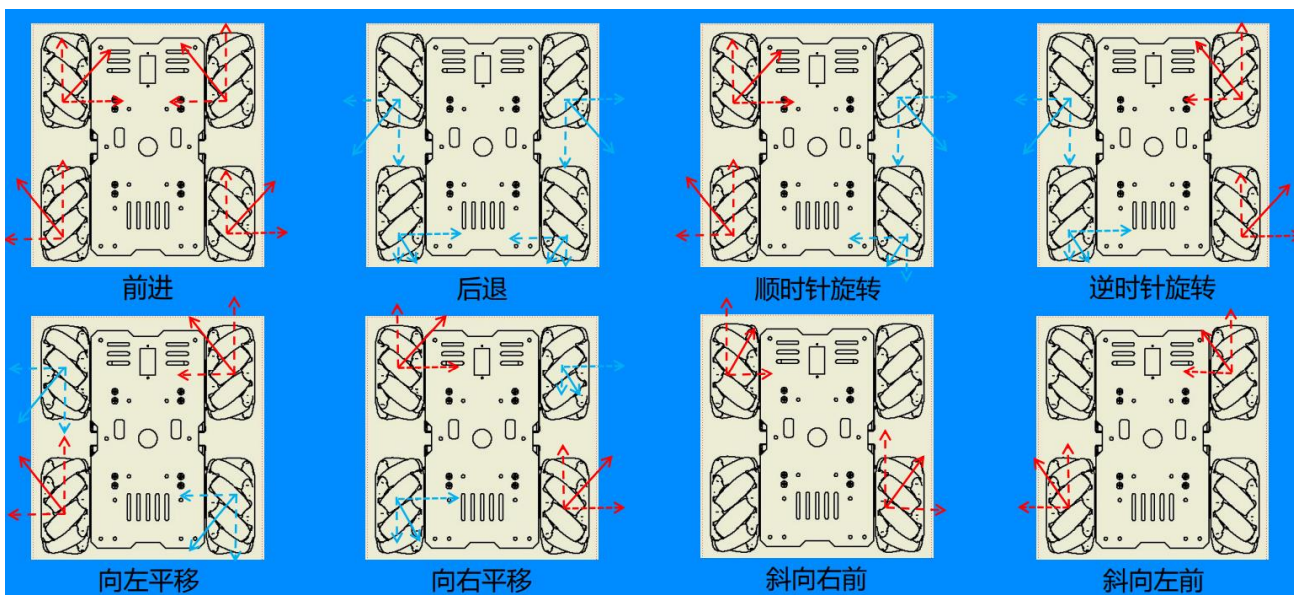


图 9.11 麦克纳姆轮运动分解示意图

下面的代码定义了电机的引脚和对应的操作命令。

```
```c++
#include <RL_ESP32_Motor.h> // 定义电机引脚
```

---

```
Motor Esp32_Motor_1(1,12,13);
Motor Esp32_Motor_2(2,14,15);
Motor Esp32_Motor_3(3,16,17);
Motor Esp32_Motor_4(4,18,19);

int speed; // 0~100
// 小车前进
void advance() {
 Esp32_Motor_1.Motor_Speed(speed);
 Esp32_Motor_2.Motor_Speed(speed);
 Esp32_Motor_3.Motor_Speed((-speed));
 Esp32_Motor_4.Motor_Speed((-speed));
}
// 小车后退
void back() {
 Esp32_Motor_1.Motor_Speed((-speed));
 Esp32_Motor_2.Motor_Speed((-speed));
 Esp32_Motor_3.Motor_Speed(speed);
 Esp32_Motor_4.Motor_Speed(speed);
}
// 小车停止
void stop() {
 Esp32_Motor_1.Motor_Speed(0);
 Esp32_Motor_2.Motor_Speed(0);
 Esp32_Motor_3.Motor_Speed(0);
 Esp32_Motor_4.Motor_Speed(0);
}
// 小车左转
void left(float degree = 0.25) {
 Esp32_Motor_1.Motor_Speed(speed);
 Esp32_Motor_2.Motor_Speed(speed);
 Esp32_Motor_3.Motor_Speed((-speed * (1 + degree)));
 Esp32_Motor_4.Motor_Speed((-speed * (1 + degree)));
}
// 小车右转
void right(float degree = 0.25) {
 Esp32_Motor_1.Motor_Speed(speed * (1 + degree));
 Esp32_Motor_2.Motor_Speed(speed * (1 + degree));
```

---

```
 Esp32_Motor_3.Motor_Speed((-speed));
 Esp32_Motor_4.Motor_Speed((-speed));
}
// 逆时针旋转
void anticlockwise() {
 Esp32_Motor_1.Motor_Speed((-speed));
 Esp32_Motor_2.Motor_Speed((-speed));
 Esp32_Motor_3.Motor_Speed((-speed));
 Esp32_Motor_4.Motor_Speed((-speed));
}
// 顺时针旋转
void clockwise() {
 Esp32_Motor_1.Motor_Speed((speed));
 Esp32_Motor_2.Motor_Speed((speed));
 Esp32_Motor_3.Motor_Speed((speed));
 Esp32_Motor_4.Motor_Speed((speed));
}
// 向左平移
void left_tran() {
 Esp32_Motor_1.Motor_Speed((-speed));
 Esp32_Motor_2.Motor_Speed((speed));
 Esp32_Motor_3.Motor_Speed((speed));
 Esp32_Motor_4.Motor_Speed((-speed));
}
// 斜向左前方
void left_oblique() {
 Esp32_Motor_1.Motor_Speed(0);
 Esp32_Motor_2.Motor_Speed((speed));
 Esp32_Motor_3.Motor_Speed(0);
 Esp32_Motor_4.Motor_Speed((-speed));
}
// 向右平移
void right_tran() {
 Esp32_Motor_1.Motor_Speed((speed));
 Esp32_Motor_2.Motor_Speed((-speed));
 Esp32_Motor_3.Motor_Speed((-speed));
 Esp32_Motor_4.Motor_Speed((speed));
}
```

---

```
// 斜向右前方
void right_oblique() {
 Esp32_Motor_1.Motor_Speed((speed));
 Esp32_Motor_2.Motor_Speed(0);
 Esp32_Motor_3.Motor_Speed((-speed));
 Esp32_Motor_4.Motor_Speed(0);
}

String messageIn;
void setup(){
 Serial.begin(115200);
 speed = 75;
 Esp32_Motor_1.mcpwm_begin();
 Esp32_Motor_2.mcpwm_begin();
 Esp32_Motor_3.mcpwm_begin();
 Esp32_Motor_4.mcpwm_begin();
}

void loop(){
 if (Serial.available() > 0) //串口接收到数据
 {
 messageIn = Serial.readString();
 if (messageIn == "FASTER")
 speed = speed + 10;
 else if (messageIn == "SLOWER")
 speed = speed - 10;
 else if (messageIn == "ADVANCE")
 advance();
 else if (messageIn == "BACK")
 back();
 else if (messageIn == "STOP")
 stop();
 else if (messageIn == "LEFT")
 left();
 else if (messageIn == "RIGHT")
 right();
 else if (messageIn == "LEFT_TRAN")
 left_tran();
 }
}
```

```

else if (messageIn == "RIGHT_TRAN")
 right_tran();
else if (messageIn == "ANTICLOCKWISE")
 anticlockwise();
else if (messageIn == "CLOCKWISE")
 clockwise();
else if (messageIn == "LEFT_OBLIQUE")
 left_oblique();
else if (messageIn == "RIGHT_OBLIQUE")
 right_oblique();
 }
}
...

```

## 22 舵机及其控制

舵机是由直流电机、减速齿轮组、传感器(可变电阻)和控制电路组成的一套自动控制系统。图 9.12 舵机示意图给出了舵机的图示。



图 9.12 舵机示意图

舵机的输入线共有三条，红色是电源线，棕色是地线，这两根线给舵机提供最基本的能源保证，主要是电机的转动消耗。另外一根线是控制信号线，一般为桔黄色。

舵机的信号线是做为输入线就是接收 PWM 信号（定时器产生）。一般 PWM 的周期是 20ms,那么对应的频率是 50hz。那么改变不同的占空比就可以控制转动的角度。其中占空比从 0.5~2.5ms，相对应的舵盘位置为 0~180 度，呈线性变化。

- \*\*示例代码\*\*

```

```c++
#include <ESP32_Servo.h>
Servo servo_25;
Servo servo_26;

```

```

void setup(){
    servo_25.attach(25,500,2500);
    servo_26.attach(26,500,2500);
}

void loop(){
    servo_25.write(45);
    delay(100);
    servo_26.write(90);
}
...

```

3 超声波传感器

图 9.13 超声波传感器示意图给出了超声波传感器的图示。由超声波发射探头发出的超声波脉冲，经媒质(空气)传到物体表面，反射后通过媒质(空气)传到接收探头，测定出超声脉冲从发射到接收所需的时间，根据传输媒质中的声速，计算从探头到物体表面之间的距离。

工作原理：

- 采用 IO 口 TRIG 触发测距，给至少 10us 的高电平信号；
- 模块自动发送 8 个 40khz 的方波，自动检测是否有信号返回；
- 有信号返回，通过 IO 口 ECHO 输出一个高电平，高电平持续的时间就是超声波从发射到返回的时间；
- 测试距离 = (高电平时间 * 声速(340M/S)) / 2。

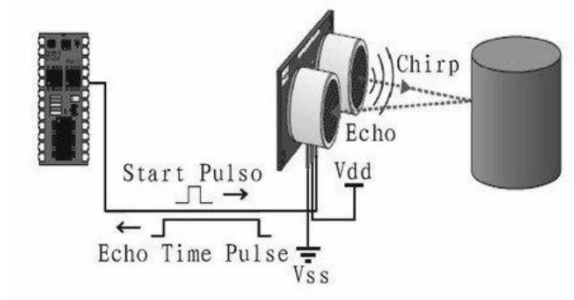


图 9.13 超声波传感器示意图

- **超声波测距示例代码**

```

``c++
const int trigPin = 4;
const int echoPin = 5;
float distance;

float checkDistance() {
    digitalWrite(trigPin, LOW);

```



```

        delayMicroseconds(2);
        digitalWrite(trigPin, HIGH);
        delayMicroseconds(10);
        digitalWrite(trigPin, LOW);

        float distance = pulseIn(echoPin, HIGH) / 58.00;    //echo time
conversion into a distance
        delay(10);
        return distance;
    }

    void setup()
    {
        Serial.begin(115200);
        pinMode(trigPin, OUTPUT);
        pinMode(echoPin, INPUT);
        distance = 0;
    }
    void loop()
    {
        distance = checkDistance();
        Serial.print(distance);
        Serial.println("cm");
        Serial.println();
        delay(1000);
    }
}

```

4 串口实现 Json 数据通信

串行接口简称串口，也称串行通信接口或串行通讯接口（通常指 **COM** 接口），是采用串行通信方式的扩展接口。串行接口（**Serial Interface**）是指数据一位一位地顺序传送。其特点是通信线路简单，只要一对传输线就可以实现双向通信（可以直接

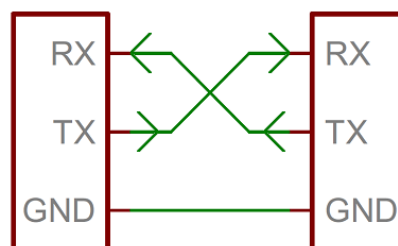


图 9.14 串口通信连接示意图

利用电话线作为传输线），从而大大降低了成本，特别适用于远距离通信，但传送速度较慢。利用串口，可以实现单片机之间、单片机与 PC 等其他设备之间的数据通信。

设备与设备之间至少需要接三个线。GND 是地线，提供低电平的参考电压。接下来就是 RX 信息接收端，还有 TX 信息发送端。单片机 A 的 RX 接单片机 B 的 TX，单片机 B 的 RX 接单片机 A 的 TX。若两个设备的 VCC 电压不一致，则也需相连。

Arduino 封装了 ESP32 的串口库函数，可直接使用并自定义引脚为手法串口，需注意的是，选择的引脚必须是双向收发的 GPI/O 口，ESP32 中有些引脚不支持双向收发 I/O，则不可用。用户需安装 Arduino_JSON 包才可调用相关功能。自动生成 Json 解析和打包代码，使用 ArduinoJson Assistant 小工具实现。用户可以利用 Arduino 串口接收并解析如下格式 Json 数据。

```
{
  "speed": [0/1/-1, 30],
  "carmove": ["STOP/ADVANCE/LEFT/...", 0.5],
  "servo": [45, 90],
  "checkdistance": 1
}
```

```
#include <HardwareSerial.h>
#include <ArduinoJson.h>
String rx_buffer = ""; //声明字符串变量
HardwareSerial mySerial1(1); // 自定义串口实例
void setup()
{
  mySerial1.begin(115200,SERIAL_8N1,32,33); //初始化并自定义 32/33 引脚为串口
}
void loop()
{
  while (mySerial1.available() > 0) // 读取串口缓冲区的数据
  {
    rx_buffer += char(mySerial1.read());
    delay(2);
  }
  if (rx_buffer.length() > 0)
  {
    DynamicJsonDocument jsonBuffer(400);
    deserializeJson(jsonBuffer, rx_buffer); // 解析 Json 数据
```

```

    JsonObject doc = jsonBuffer.as<JsonObject>();
    int speed_0 = doc["speed"][0]; // 0/1/-1
    int speed_1 = doc["speed"][1]; // 30
    const char* carmove = doc["carmove"][0]; // "STOP/ADVANCE/LEFT/..."
    float degree = doc["carmove"][1]; // 0.5
    int servo_0 = doc["servo"][0]; // 45
    int servo_1 = doc["servo"][1]; // 90
    int checkdistance = doc["checkdistance"]; // 1
    rx_buffer = "";
  }
}

```

- **ArduinioJson 发送 Json 数据**

/*

利用 ArduinoJson 发送如下格式 Json 数据：

```

{
  "speed": [0/1/-1, 30],
  "carmove": ["STOP/ADVANCE/LEFT/...", 0.5],
  "servo": [45, 90],
  "checkdistance": 1
}
*/

```

```

#include <ArduinoJson.h>

```

```

void setup() {
  Serial.begin(115200);
  Serial.println("begin now");
}

```

```

void loop() {
  DynamicJsonDocument doc(256);

  JsonArray speed = doc.createNestedArray("speed");
  speed.add(0/1/-1);
  speed.add(100);
  doc["carmove"][0] = "STOP/ADVANCE/LEFT/...";
  doc["carmove"][1] = 0.5;
}

```

```

JSONArray servo = doc.createNestedArray("servo");
servo.add(45);
servo.add(90);
doc["checkdistance"] = 1;

char json_string[256];
serializeJson(doc, json_string); // 打包成 Json 串
Serial.println(json_string); // 串口输出
}

```

9.3.3

小车代码分析

1. 小车功能结构

智能小车通过摄像头感知周围环境自主地进行运动控制，采集环境数据后在开发者套件上进行 AI 推理，根据推理结果发出指令控制小车的运动状态。小车通过 ESP32 微控制器实现运动状态的控制，使用 Arduino 平台可以对其进行嵌入式开发。主控与小车主体间需要通过串口协议进行双向通信，实现控制指令的发出和数据的返回。图 9.15 智能小车结构图给出了小车的整体架构图。

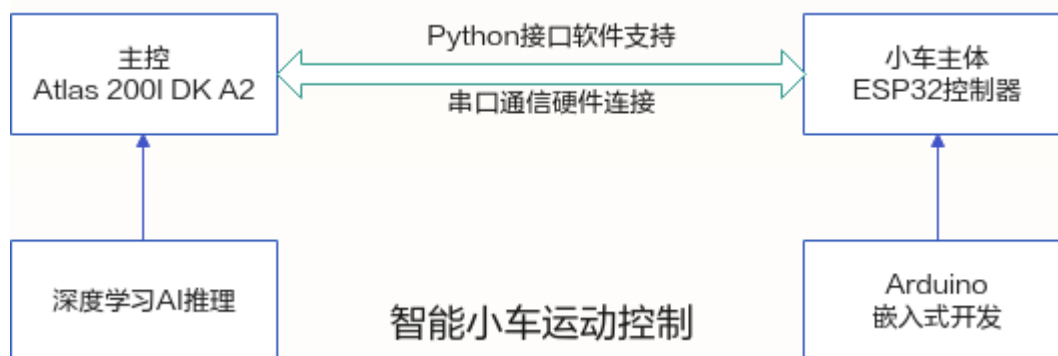


图 9.15 智能小车结构图

本项目涉及智能小车控制底层代码、简单运动和复杂运动代码、各类工具以及多个推理模型。开发板配套开源代码的工程目录如下图 9.16 智能小车源代码结构所示。

```

Car
├── python
│   ├── main.py    #小车 demo 运行总入口
│   ├── requirements.txt #小车 demo 样例所需依赖
│   └── src

```

```

|   |   |—— Lane-Follow-Train #循迹驾驶部分训练代码
|   |   |—— det_label_spilt #检测辅助分类模型
|   |   |—— actions
|   |   |   |—— base_action.py #小车基础运动代码
|   |   |   |—— complex_actions.py #小车复杂运动代码
|   |   |   |—— __init__.py
|   |   |—— models
|   |   |   |—— bsae_model.py #基础模型
|   |   |   |—— det_cls.py #检测分类模型
|   |   |   |—— __init__.py
|   |   |   |—— lfnet.py #推理
|   |   |   |—— yolov5.py #yolov5 检测模型
|   |   |—— scenes # 智能车预设场景相关代码
|   |   |   |—— base_scene.py
|   |   |   |—— command.py
|   |   |   |—— helper.py
|   |   |   |—— __init__.py
|   |   |   |—— lane_following.py
|   |   |   |—— manual.py
|   |   |   |—— tracking.py
|   |   |—— utils #工具类 python 文件
|   |       |—— acl_utils.py
|   |       |—— camera_broadcaster.py
|   |       |—— common_utils.py
|   |       |—— constant.py
|   |       |—— controller.py
|   |       |—— cv_utils.py
|   |       |—— __init__.py
|   |       |—— init_utils.py

```

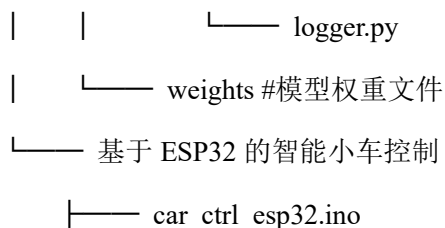


图 9.16 智能小车源代码结构图

注意：本书提供给用户的代码包含两类，第一类是开发板配套开源代码，除特别标注，9.3.3 节所述代码为开源代码框架；第二类为用户代码，由本书作者在根据教学实践进行调整、完善。用户在确定方案前需要通读本书相关部分资料、通览本书提供的各类代码并根据任务要求调研对应的技术路线、主流实现方案的基础上制定本组的实现方案，在编写或选用现有软件框架的基础上进行功能实现。

2. 小车代码代码解析

1) main.py

`main.py` 在源码下的 `ascend-devkit/src/E2E-Sample/Car/python` 路径，其运行流程为初始化控制器、连接硬件、初始化相机、开进程启、创建消息队列、传递按键/指令、根据 `mode` 启动对应任务、按 `ESC` 退出、停止所有进程并停车，图 xx 智能小车主程序流程图给出了完整的代码流程。代码通过指定解析不同的 `mode` 参数(`python main.py--mode=...`)来调用不同的场景接口。程序中通过如 `Argument Parser` 命令行参数解析器设定了四种模式 `choices=['cmd', 'voice', 'manual', 'easy']`。其中 `manual` 对应手动控制控制模式，用户通过键盘控制小车，按键发送到消息队列，按 `ESC` 停止。`cmd` 对应命令行控制模式，用户可输入指令启动场景，如 `line_follow /avoid_obstacle` 等，该模式支持 `stop / clear` 命令，可同时运行多个场景。`Easy` 对应简易自动模式，系统同时启动两个任务，其中 `Helper` 任务对应辅助、识别功能，`LF` 任务对应巡线（`Line Follow`）功能，按下 `ESC` 后两个任务全部停止。`Manual` 负责手动控制底盘运动，`Easy` 负责自动控制；这两种模式均通过共享内存空间的方式读取摄像头的信息。

`main.py` 实现了跨进程通信。`Queue(msg_queue)`实现了主线程和各场景子进程通信，进行键盘按键、控制指令的传递。共享内存使得多进程共享摄像头画面，避免多开摄像头冲突、降低性能开销的问题。

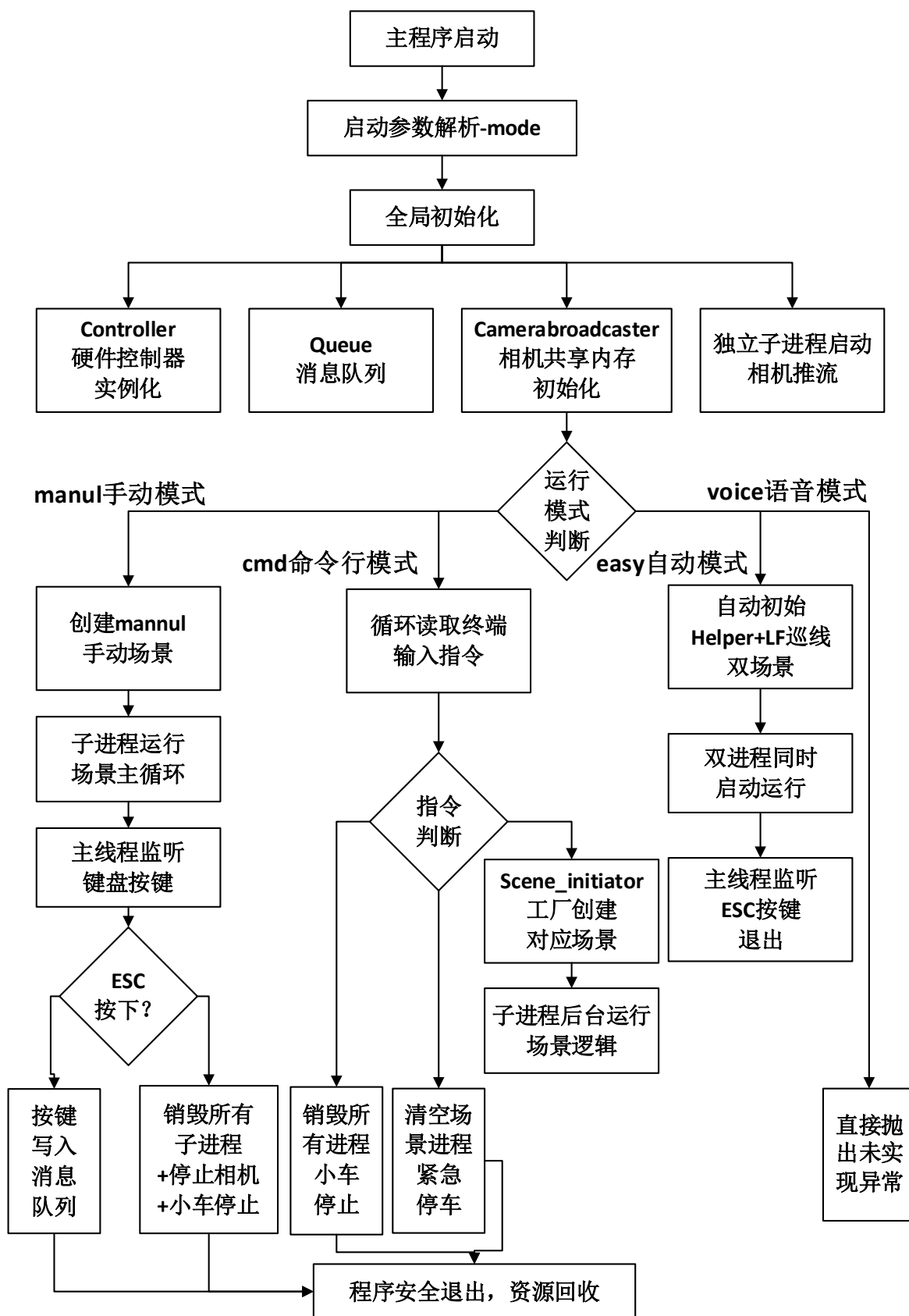


图 9.17 智能小车主程序流程图

2) manual.py

main.py 的 manual 模式只开启了一个进程，用于实现手动控制。在 ascend-devkit\src\E2E-Sample\Car\python\src\scenes 路径中找到 manual.py，即手动模式的底层函数，通过阅读代码我们可以知该模块通过监听键盘指令消息队列，实现小车的全手动远程控制，包括基础运动、速度调节、舵机控制、摄像头截图、自定义动作和复杂动作执行。

该模块首先进行类实例化，调用父类 BaseScene 的构造方法，初始化内存、摄像头信息和消息队列，设置小车默认运行速度为 29，创建截图保存目录./capture（不存在则自动创建）。然后进行场景初始化，执行 init_state()方法，发送舵机控制指令，将摄像头云台复位到初始角度，若初始化失败，记录错误日志并直接退出场景。初始化成功后，创建与摄像头共享内存缓冲区绑定的 frame 数组，记录场景启动日志，初始化 last_action 为舵机复位动作，进入无限消息监听循环，持续等待键盘指令。在循环中，通过读取键盘按键来执行 base_action 中的各种动作，或进行拍照。

表 9.xxx 小车手动控制动作表给出了小车的动作控制按键及对应的功能、分类。

表 9.1 小车手动控制动作表

按键	功能	动作类型
↑ / ↓	速度调节（25~60 范围）	无动作，仅修改全局速度
← / →	左右平移	基础动作
w/a/s/d	前进/左转/后退/右转	基础动作（左转/右转带转向角度 1.1）
q/e	逆时针/顺时针原地旋转	基础动作
space	紧急停止	基础动作
esc	停止并退出手动模式	系统动作
c	保存当前摄像头截图到./capture 目录	系统动作
t/r	执行预设自定义电机动作	自定义动作
z	执行掉头动作	复杂动作
x	执行自旋动作	复杂动作

helper.py

ascend-devkit\src\E2E-Sample\Car\python\src\scenes 路径下的 helper.py 实现 main.py 函数中 easy 模式下的地标识别任务。

该代码首先进行场景初始化，始化共享内存名称、摄像头参数和消息队列；初始化核心成员变量 self.det（YOLOv5 目标检测器实例）为 None。接着为进入场景主循环进行准备工作，记录场景初始化开始日志、构建 YOLOv5 离线模型文件(*.om)路径、检查模型文件是否存在并加载、记录模型初始日志和舵机硬件初始化。完成该准备工作后，代码进入场景主循环，先调用 init_state() 进行执行判断，然后创建与摄像头共享内存缓冲区直接绑定的 frame 数组，避免数据拷贝，提升帧率；代码同时初始化 last_action 变量用于记录上一次执行的动作类别，实现动作防抖，同时进行异常捕获块，开始主循环。

在主循环阶段，每轮循环一帧摄像头数据，执行一次完整的检测与控制流程，其首先进行全局状态检查，对共享变量 self.stop_sign.value、self.pause_sign.value 进行检查，如果为 True，跳过本次循环剩余逻辑，继续下一轮检查。然后记录当前时间戳 start，通过 img_bgr = frame.copy() 进行当前摄像头帧数据复制工作。接着代码调用 self.det.infer(img_bgr)，将复制的帧传入 YOLOv5 检测器，进行目标检测推理，得到检测结果 bboxes。bboxes 通过格式[x1, y1, x2, y2, cate, score]输出每个目标框的左上角坐标、右下角坐标、类别名称、置信度，获得的 bboxes 按置信度 score 从高到低排序。代码接着遍历处理第一个符合条件的检测目标，计算当前目标的中心坐标(x, y)和宽高(h, w)，记录检测到的目标类别日志。为避免多目标同时出现时的频繁动作切换，保证行驶稳定性，代码进行了防抖设计。最后，代码按照检测类别进行动作触发，表 9.xxx 分类动作表给出了不同类别标志的对应动作。执行对应动作后，立即 break 跳出目标遍历循环，进入下一轮主循环。

表 9.2 分类动作表

标志类别	触发区域	执行动作	动作后等待时间
left（左转）	x: 300~1000, y ≥ 450	TurnLeftInPlace ()（原地左转）	1 秒
right（右转）	x: 420~950, y ≥ 580	TurnRightInPlace ()（原地右转）	1 秒
turnaround （掉头）	x: 420~800, y > 350	TurnAround ()（原地掉头）	2 秒

当 self.stop_sign.value 变为 True 时，跳出主循环，场景结束，如果捕获 KeyboardInterrupt 异常

（用户按下 Ctrl+C），则立即执行 Stop()动作停止所有电机，防止小车失控

注意：当前代码没有显式的资源释放逻辑（如卸载 YOLO 模型、释放 NPU 资源），依赖 Python 的垃圾回收机制自动处理。

lane_following.py

main.py 函数中的 easy 模式开启的第二个进程，用于巡线任务。在 ascend-devkit\src\E2E-Sample\Car\python\src\scenes 路径下的 lane_following.py 找到了巡线任务的实现细节。该部分实现视觉感知、AI 推理、动作决策和硬件执行功能。代码中摄像头采集道路图像被送入 LFNNet 模型推理得到转向参数，进而控制小车前进、左转、右转，实现小车在车道中心行驶。整个过程分为以下几个部分。

首先是场景初始化阶段（__init__），该阶段代码调用父类 BaseScene 初始化，获取共享内存、摄像头信息、消息队列。定义 AI 模型对象 self.net 初始为空。设置小车巡线默认直行速度。其次是状态与资源初始化（init_state），代码打印日志，标记 LF 场景开始初始化；加载 AI 模型文件 weights/lfnet.om，若文件不存在则报错并退出初始化；初始化巡线视觉模型 LFNNet 用于车道偏离检测；调用控制器执行舵机复位，将云台角度设置为初始值，使摄像头保持朝前。再次是主循环启动（loop），代码调用 init_state 完成模型与硬件初始化，若失败直接退出；从共享内存中映射摄像头图像数据，实现实时读取画面；打印日志，标记巡线主循环开始；发送启动指令，让小车进入运行状态。最后是进入实时巡线主循环（while True），在这个阶段需要进行状态检测，如有则根据收到的停止、暂停信号执行对应动作；若无则进行图像获取，复制当前一帧摄像头图像送入 LFNNet 模型进行推理，得到转向控制值 curr_steering_val，该值的范围是为 0~135。需要根据不同的数值进行对应的操作，90 表示位于车道中心，无需操作；小于 90 表示小车偏右，需要左转修正；大于 90 表示小车偏左，需要右转修正。根据不同的偏离值，可以分区间设置转向动作的数值，实现分区控制。当用户按下 Ctrl+C 中断程序时，自动执行 Stop()函数，停止电机输出，实现停车动作。

Tracking.py

Tracking 标追踪模块通过继承 BaseScene 基类实现智能小车的视觉自主跟随功能，整体依靠 YOLOv5 模型完成目标检测与运动控制；在 __init__ 构造函数中完成类成员变量的初始化与父类构造，在 init_state 初始化函数中加载 weights 目录下的 tracking.om 离线模型，并通过控制器执行 SetServo 动作将舵机设置为朝前的初始角度，初始化完成后进入 loop 主循环函数开始持续运行追踪逻辑；在 loop 主循环内，程序从共享内存中读取摄像头实时图像，调用 self.model.infer 函数完

成目标检测推理并获取检测框 `bboxes`，若未检测到目标则执行 `Stop` 停车动作，若检测到多个目标则按置信度排序选取最优检测框，随后计算检测框中心坐标 `x`、`y` 与宽高 `w`、`h`，根据检测框面积与垂直位置动态选择高速或低速前进速度，再依据中心 `x` 坐标判断目标位置并执行 `TurnLeft`、`TurnRight` 或 `Advance` 动作，同时判断目标是否过近，若检测框面积过大或垂直位置过低则执行 `Stop` 停车指令，最后仅在动作发生变化时通过 `self.ctrl.execute` 函数下发控制指令，从而实现目标检测、距离判断、速度调节、方向纠偏及安全停车的完整目标追踪流程。

`base_action.py&complex_actions.py`

`base_action.py` 与 `complex_actions.py` 共同构成智能小车的运动动作库，二者分层协作，实现基础运动与组合运动的统一管理。其中 `base_action.py` 作为底层基础动作模块，以抽象基类 `BaseAction` 为核心，封装了小车所有原子化运动逻辑。模块通过 `motor_rating` 电机校准系数补偿硬件差异，保证四电机转速均衡；各类基础动作重写速度生成方法，通过返回不同电机转速列表，分别实现前进、后退、转向、平移、原地旋转、舵机控制、延时等单一动作，最终输出电机与舵机的控制数值，仅完成运动参数计算，不参与串口通信。`complex_actions.py` 基于基础动作构建复合动作，继承抽象类 `ComplexAction`，依靠 `action_seq` 动作序列列表，将多个基础动作、延时动作按顺序组合，实现原地转向、掉头、平稳启动、自动泊车等复杂连贯动作。运行时控制器会依次解析并执行序列内的每一个原子动作，最终将计算得到的控制指令下发至下位机。两个模块分工明确，基础动作提供最小运动单元，复合动作完成运动流程编排，为巡线、遥控等上层场景提供统一的动作调用接口，保障小车运动控制灵活、稳定。

`controller.py`

`Controller` 模块是智能小车运动控制系统的核心调度与硬件通信中心，所有上层场景（如自动巡线）、动作模块（基础动作、复合动作）最终都通过该模块与底层 `ESP32` 硬件交互。它通过单例与文件锁保证多进程安全运行，负责接收动作指令、计算运动参数、封装串口协议、通过 `UART` 发送控制指令，并同步全局运动状态。该模块实现了上层逻辑与底层硬件的解耦，为小车提供稳定、可靠、实时的运动控制能力。

在前面的流程中，场景模块的巡线逻辑中通过模型推理得到转向值后由程序判断需要执行前进、左转、右转等动作；代码中根据推理结果，实例化对应的动作类后调用 `self.ctrl.execute(action)` 将动作交给控制器处理；进入 `Controller.execute()` 后，控制器获取文件锁，保证多进程安全，从序列化文件同步最新速度、舵机角度；然后进行动作解析与计算，代码遍历动作序列，调用 `action()` 方法，由 `base_action.py` 计算出 4 个电机速度 + 2 个舵机角度，最后是数据组包与串口发送，控

制器将 6 个控制数值+校验位拼接为 7 个数值，编码为 14 字节小端有符号数据，通过 UART 串口发送给 ESP32。ESP32 接收到数据后驱动电机，控制器读取返回结果，更新状态并保存到序列化文件，实现多进程同步。流程结束后释放锁，动作执行完成。

3. 测试前注意事项

1) 关于模型

前面的代码中为直接使用训练好的模型，本书所涉开发板开源代码使用的端到端的训练方法，需要自行采集图片并进行角度标注后和训练，开源代码中的 Lane-Follow-Train 文件夹用于端到端训练巡线模型的工程。本书还提供了基于车道线识别的 LaneNet 车道线分割拟合方案，该方案需要在车道线识别的基础上进行角度计算以进行下一步的动作控制。这两部分将在本章后续小节进行说明。

2) 关于小车电气控制

在 base_action.py 中，需要注意 self.motor_rating 涉及到上位机发给电机驱动板四个电机的转速大小。每个基础运动函数返回的列表值，主要用于修改当前运动的电机转动的方向和转速大小。在开始进行 demo 测试前，需要通过手动模式调整代码的这个参数，使得四个轮子的转速、方向一致（由于机械、电气结构差异，不同小车都需要调整此参数）。匹配好基础参数也会提升后续 pid 控制参数匹配的便利性。

3) 关于 pid 闭环控制

本书所述开源代码使用基于视觉的直接转向控制，逐帧采集图像、计算偏移角度、进行姿态修正，无连续轨迹规划、无行走状态判断，底层电机使用简明的直驱电机。整个方案存在容易实现的优点，同时存在实现效果一般、较难进行精细的状态控制的问题。为取得较好的控制效果，建议在边缘控制板对车身偏移角度进行 pid 调整，优化巡线纠偏和路径跟随功能。同时，为了提升车身偏移角度的 pid 控制效果，采用带编码器的电机，对各个轮子的转速实现 pid 控制，弱化因为电池电量波动、车道摩擦系数变动和坡度变化导致的速度失真给上层车身偏移控制带来的干扰。结合现有的开源方案，可以循序渐进的采用以下两种方案进行优化。

方案 1 在边缘开发板增加角度环，对 LF/LaneNet 推理获得的角度进行 pid 控制后下发；在 ESP32 控制底板增加速度环式：一是分工式串级 PID，上位机运行航向角度外环 PID，ESP32 运行电机速度内环 PID，二者串联协作，兼顾算法灵活性与控制实时性。

方案 2 采用全底层 PID，上位机仅输出视觉偏差，将推理夹角通过串口通信传给驱动底板

后，完整 PID 算法在底板单片机运行，实时性最优。

9.3.4 智能小车测试

采用前面章节的所使用的 Mobarxterm 工具 SSH 登录到开发板，并进入用户目录后，下载本书所涉源代码，将 src/E2E-Sample/Car 目录下与智能车有关的代码文件下载到开发板用户目录下，进入#进入到代码所在目录。

```
cd ~E2ESamples/src/E2E-Sample/Car/python
```

安装必要的依赖文件：

```
pip3 install -r requirements.txt
```

运用文件夹下的 main.py 文件，并设置模式为手动模式：

```
python3 main.py --mode manual
```

代码开始运行后，可以在用户界面上输出，图图 9.18 demo 运行输出图给出了对应的输出。

```
(base) root@davinci-mini:/home/HwHiAiUser/E2ESamples/src/E2E-Sample/Car/python# python main.py
[EVENT] PROFILING(5821,python):2023-03-27-02:24:50.860.329
[msprof_callback_impl.cpp:236] >>> (tid:5821) Started to register profiling ctrl callback.
2023-03-27 02:24:52 [MainProcess:5821][INFO]: start
2023-03-27 02:24:52 [Process-2:5841][INFO]: update_controller_speed: True
2023-03-27 02:24:52 [Process-2:5841][INFO]: self.speed: 0
2023-03-27 02:24:52 [Process-2:5841][INFO]: [0, 0, 0, 0, 90, 65, -12345]
2023-03-27 02:24:52 [Process-2:5841][INFO]: action SetServo execute SUCC
2023-03-27 02:24:52 [Process-2:5841][INFO]: Manual loop start
```

图 9.18 demo 运行输出图

图 9.2 智能小车控制键位表

键位	动作说明	键位	动作说明
w	前进	e	顺时针旋转
a	左转	↑	加速（小车运动状态与静止状态均可以调整）
s	后退	↓	减速（小车运动状态与静止状态均可以调整）
d	右转	c	捕获当前摄像头的图片
q	逆时针旋转	space	暂时停车

可以用键盘输入，手动控制小车的各种移动。表 9.2 智能小车控制键位表给出了不同按键对应的功能。

注意：此处使用的模型较为简单，且因为不同小车的个体特征、光线问题等多种原因导致小车的运行效果一般为正常现象。

需要结合代码框架和本书在模型训练、闭环控制等各个方向进行优化、调整，以实现任务要求。

其中模型训练可参照/Car/src/Lane-Follow-Train 中的步骤说明标注并扩充数据集，再使用训练代码训练模型。

训练完成后在./output 中输出 onnx 模型，使用 atc 模型转换命令 进行转换。

```
atc --model=./lfnet.onnx --framework=5 --output=./out/lfnet --soc_version=Ascend310B1
```

将./out/lfnet.om 放入到 Car/python/weights 中，或替换原有循迹 om 模型进行使用。

若需要训练除停车标外的其他标识作为停车检测物，在使用[开发者套件模型适配工具](https://ascend-repo.obs.cn-east-2.myhuaweicloud.com/Atlas%20200I%20DK%20A2/DevKit/tools/23.0.RC1/1.1.3/Ascend-devkit-model-adapter_1.1.3_win-x86_64.exe)构建了检测部分模型后，参照 det_label_split 部分构建分类辅助模型，再转换 om 模型。

运行如下在命令行输入 Tracking，小车即可进入追踪模式

```
python3 main.py -mode==cmd
```

若需要重新训练不同停车标识或追踪目标，可以使用[开发者套件模型适配工具](https://ascend-repo.obs.cn-east-2.myhuaweicloud.com/Atlas%20200I%20DK%20A2/DevKit/tools/23.0.RC1/1.1.3/Ascend-devkit-model-adapter_1.1.3_win-x86_64.exe)构建目标检测应用。停车场景中检测+分类模型分别命名为'yolo.om'与'cls.om'并放入到 Car/python/weights 中;追踪模式的单检测模型命名为'tracking.om'并放入到 Car/python/weights 中做替换即可。

9.4 模型应用

模型训练是深度学习视觉算法落地的核心环节，直接决定目标检测、车道线检测模型的识别精度、实时性与场景泛化能力，也是本智能小车实现车道循迹、行人检测、人行道识别、路口转弯及原地掉头等功能的基础。本项目针对两类视觉任务分别完成数据集构建、标注、训练、数据增强与模型迭代，并结合昇腾嵌入式硬件完成模型适配，下面分别对目标检测、车道线检测的训练流程、技术要点与方案选型进行综述。

9.4.1 目标检测概述

在目标检测模型训练方面，本项目采用 YOLOv5 算法统一实现行人、人行道、转弯标识、掉头标识等多类目标检测任务。数据集采集依托车载摄像头完成实地拍摄，覆盖正常光照、逆光、阴影、目标遮挡等多种实际工况，按照 8:1:1 的比例划分为训练集、验证集与测试集。标注工作使用 LabelImg 工具完成，针对行人、人行道、交通标识等不同类别逐一标注外接矩形框与类别标签；为提升模型鲁棒性，训练前对数据集执行随机裁剪、图像翻转、色域变换、马赛克混合等数据增强操作，丰富样本多样性。训练过程基于 PyTorch 框架开展，选用轻量化网络结构控制模型参数量，适配边缘设备算力；训练完成后依次执行模型导出、ONNX 格式转换、昇腾 ATC 工具离线量化，最终生成 .om 格式推理模型部署至 Atlas200I DK A2 开发板。相较于传统手工特征检测算法，基于深度学习的 YOLOv5 经过充分训练后，对形态多变、存在遮挡的行人与各类标识具备更强的识别能力，同时兼顾推理速度，完全满足小车行驶过程中的实时检测要求。传统 Viola-Jones、HOG+SVM 等算法因泛化性差、实时性不足，本项目仅作为技术参考，未投入实际训练与使用。

9.4.2 车道线检测

在车道线检测模型训练上，本项目提供两套技术方案供按需选择，分别为 LaneNet+H-Net 分割拟合方案与 LF 端到端角度回归方案，两类方案的数据集制作、训练逻辑与应用场景存在差异。两类视觉模型的训练均围绕精度、速度、硬件适配、场景泛化四大目标开展。目标检测模型以多目标识别、高实时性为训练核心；车道线检测则根据应用复杂度区分两种训练路线，复杂全场景优先选用 LaneNet+H-Net，简易固定场景优先选用 LFNet。合理的数据集构建、数据增强与模型轻量化训练策略及推理调优，是两类算法能够在 Atlas200I DK A2 嵌入式平台稳定运行的关键，也为小车多场景自主行驶提供了可靠的视觉感知支撑。下面我们分别对两个方案进行说明。

1 LaneNet 方案，

算法源自经典车道检测论文，将车道识别定义为实例分割任务。训练采用 TuSimple 标准车道数据集及本地自制数据集，使用 Labelme 工具以逐点连线方式标注车道坐标，标注格式遵循公开数据集规范。

如图 9.19 LaneNet 车道线检测流程图所示，LaneNet 将每条车道线作为一个独立实例，支持可变数量车道线检测和变道场景。该算法其采用双分支结构联合训练，其中分割分支利用交叉熵损失区分车道与背景，嵌入分支依靠聚类损失完成不同车道实例区分，配套的轻量级 H-Net 网络则以车道拟合误差为损失函数，学习自适应透视变换参数，解决固定视角在坡道、起伏路面拟合失效的问题。该方案训练流程完

整，模型可输出车道坐标、斜率、曲率等多维度信息，经过训练后能够适配多车道、弯道、坡道、虚线及车道遮挡等复杂场景，可同时支撑循迹、转弯、掉头等全功能场景。

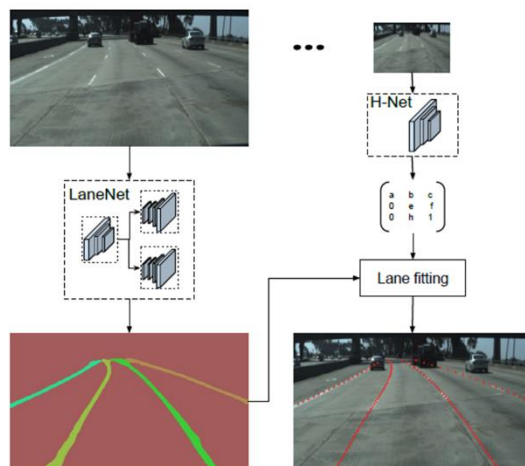


Fig. 1. System overview. Given an input image, LaneNet outputs a lane instance map, by labeling each lane pixel with a lane id. Next, the lane pixels are transformed using the transformation matrix, outputted by H-Net which learns a perspective transformation conditioned on the input image. For each lane a 3rd order polynomial is fitted and the lanes are reprojected onto the image.

图 9.19 LaneNet 车道线检测流程图

代码中，摄像头拍摄的 RGB 图像（512x256 像素）被通过共享编码器把图像压缩成高级特征图。然后采用多任务学习方法、通过两个分支实现了不同任务的输出。首先是共享编码器，采用 Enet 实现实时网络分割，两个分支任务共享同一个底层特征提取器，然后分割任务采用分割解码器依据二值掩码检测是否有车道，嵌入任务采用嵌入解码器依据 4 维嵌入判断属于哪一个车道？

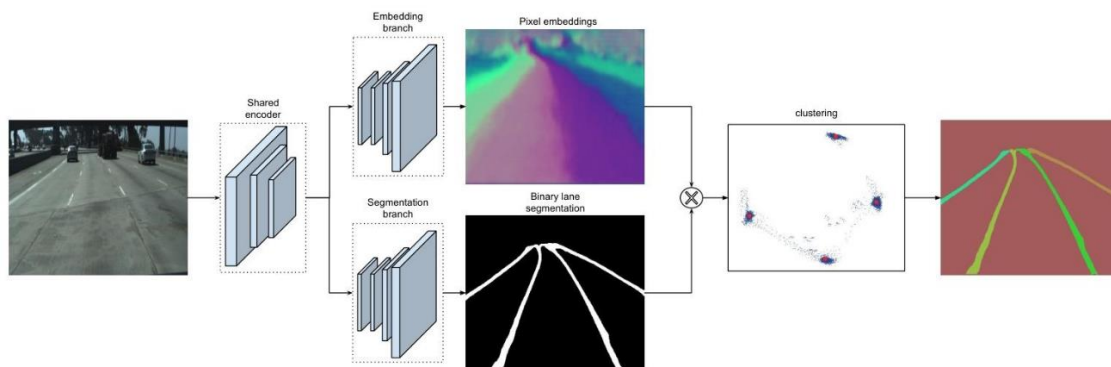


图 9.20 LaneNet 车道线检测流程图

LaneNet 最核心的创新就是这个判别损失函数（**Discriminative Loss**）。它用非常优雅的数学形式，实现了 "同簇靠近、异簇远离" 的训练目标。

$$L = L_{var} + L_{dist}$$

其中：

$$Lvar = \frac{1}{C} \sum_{CA=1}^C \frac{1}{N_c} \sum_{i=1}^{N_c} [\|\mu_c - x_i\| - \delta v]_+^2$$

$Lvar$ 表示簇内拉力，将车道线相邻的像素点汇聚。

$$L_{dist} = \frac{1}{C(C-1)} \sum_{CA=1}^C \sum_{CB \neq CA} [\delta_d - \|\mu_{ca} - \mu_{CB}\|]^2$$

L_{dist} 表示簇间推力，实现不同车道线的分离。算法通过合适的参数设计，实现了车道线像素点的聚类。

开源代码算法通过对推理结果的多项式拟合，实现了车道线的单条识别和多条分离。下面是拟合公式：

$$x = ay^3 + by^2 + cy + d$$

将开源算法自带的预训练模型应用到我们的地图上时，效果不佳，如下图 9.21 所示，具有非常多的噪声。可以基于规则进行后处理，将噪声直接过滤；也可以微调模型，使用本地采集的图像增强模型在本地图下的检测能力。

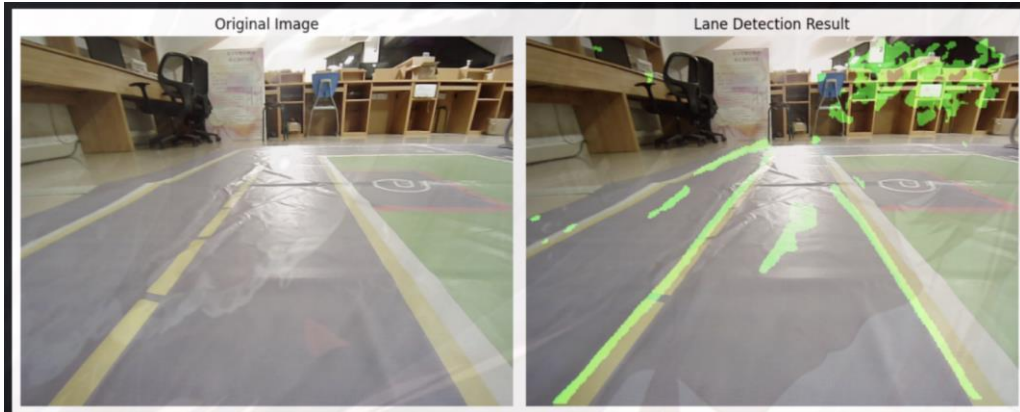


图 9.21 预处理模型在本地图场景下表现不佳

本方法使用的数据集使用 labelme 工具进行标注，使用 Create Line 方法进行两点一线标注形式，得到 json 格式的标注文件。

模型训练与测试，根据需要改写并运行开源代码中的 Train.ipynb 文件内容，然后利用 test.ipynb 文件进行模型测试（修改后的 test 文件后处理已经完善，可以看到拟合出来的直线），如图 9.22 所示微调训练及结果示意图。

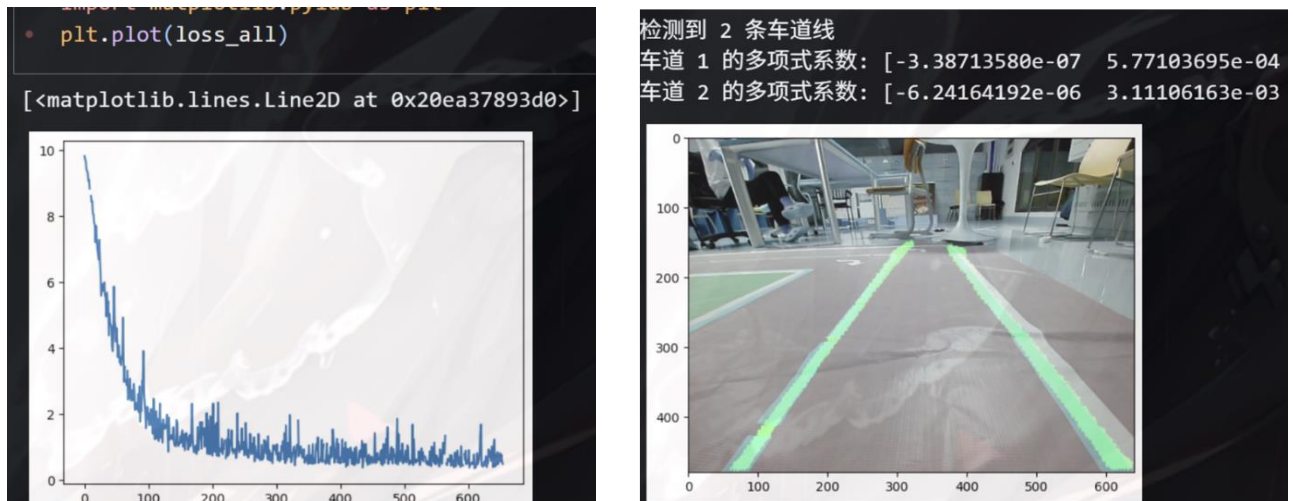


图 9.22 微调训练及结果示意图

训练得到的模型需要将其格式转换为 onnx 模型，然后将 onnx 模型转换为 om 模型。我们可以运行 transform.py 文件，将 model 模型转 onnx 模型，然后在昇腾 NPU 服务器上参考下面的 ATC 转换命令将 onnx 模型转换为 om 模型，开发板也可以转换，但是需要时间较长且容易出错。

```
atc --model=lanenet_model_1.onnx \
    --framework=5 \
    --output=modelB_0 \
    --soc_version=Ascend310B4 \
    --input_shape="input_image:1,3,256,512"
```

2 LNet 端到端回归方案

该方案摒弃车道分割、曲线拟合等中间环节，训练时直接以道路图像作为输入，以小车最优转向角度（0~135）作为标签，搭建轻量级卷积网络学习图像与转向角度的非线性映射关系。数据集由小车在实际赛道采集并逐图标注角度标签，训练过程简单、收敛速度快，模型体小、训练成本低。但该模型性能高度依赖训练样本的覆盖范围，仅在训练场景内表现稳定，面对陌生路况、大幅弯道时精度会明显下降，更适用于固定简易赛道的训练与部署。

Lane-Follow-Train 文件夹用于端到端训练巡线模型的工程。config.yaml 用于配置训练用的各种参数。其中 dataset_path 用于配置训练集的输入路径；output_dir 用于配置模型文件的输出路径；width 和 height 指定输出的维度；device:指定训练使用 CPU 或 GPU，默认为 CPU。修改为 cuda:0 则使用 GPU；epoch_num 指定训练轮次；batch_size 指定处理一批数据的数据量；lr 指定学习率；to_onnx 指定是否将输出模型转换为 onnx 模型（原始模型为.pth 模型）；onnx_model_name 指定输出模型的名称。

数据集传入 config 制定的数据集路径 dataset 下，包括一个 images 文件夹储存图像，一个 label.txt 文件储存标签。train.py 为训练代码，由于基本配置在 config 中已经指定，因此对于 train.py 仅需关注一点。即 model=resnet18()说明训练用模型为 resnet 模型。export.py 用于将出来的 pth 模型转换为 onnx 模型，并在 train.py 中调用。

需要注意的是，该算法的图像采集需要匹配开源代码提供的算法，通过程序实时采集舵机的转向角度和对应的图片后进行保存，实现采集、标注过程的同步完成。考虑到车子运行过程中的各类误差及误差的累积，存在记录的转向角和实际情况存在偏差的问题，导致训练出来模型的效果不佳。

3 opencv 车道线检测

在入门级的人工智能运用中，也常可以看到基于 opencv 的车道线检测应用。

如果使用该方法，为了排除远方噪声干扰（例如桌子、地板等），首先裁剪出一段 ROI 区域，让小汽车只关注近处的地图段。车道线的颜色为黄色，与地图其它部分差别明显，可以作为一个特征进行过滤。找到当前视角下适合的 RGB 阈值，对原始图像进行掩码，在保留黄色车道线的前提下尽可能减少噪声。通过侵蚀操作消除仍然存在的噪点。由于第 2 步操作和侵蚀操作很容易对车道线某部分“截断”，因此，通过膨胀操作将断连的线连接起来。使用霍夫变换检测直线，调整好霍夫变换检测直线的参数，使得其能够得到唯一的直线段， 如下图 9.6.1 所示

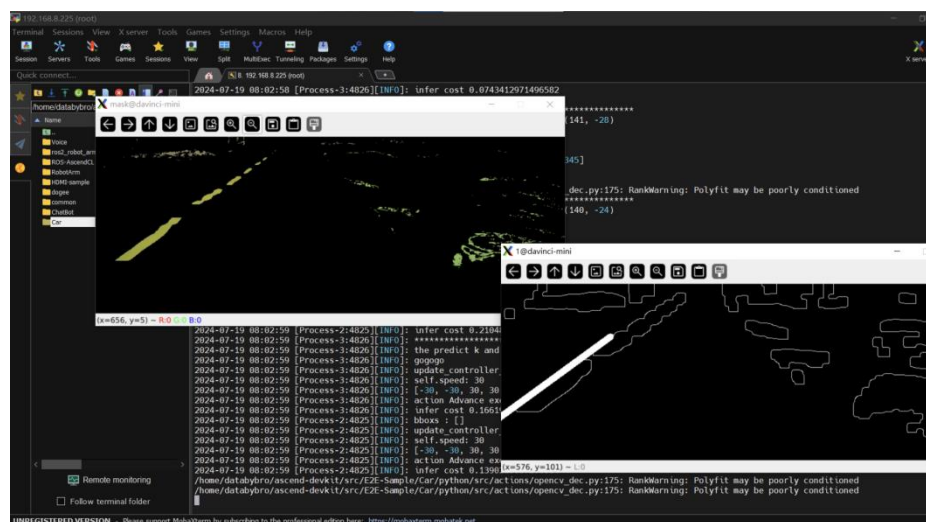


图 9.23 opencv 车道线检测，左图是 ROI+RGB 过滤结果，右图是检测结果

该方法的整个流程都是硬编码的，尤其依赖 RGB 阈值、侵蚀膨胀操作的次数和核大小、霍夫变换参数等手动调整的参数值，一旦光线等环境条件发生变换，整套参数都会失效，鲁棒性非常差，建议读者不姚使用该方法。

9.4.3 地标检测

为了实现地标检测功能，需要选用分类模型，检测地标的转向类别以及像素坐标，这里以 yolov5 为例介绍，这里使用标注工具 labeling 进行标注，建议数据集划分按 8:1:1 进行，建议在光照强度、位置平移、裁剪等维度进行数据增强，或者在不同光照条件下进行图像采集。yolov5 训练步骤详见第六章相关部分。图 9.23 给出了验证集的训练结果可视化。目标检测模型也需要进行模型转换后使用。



图 9.23 验证集结果可视化

9.4.4 决策系统

经过之前的步骤，我们已经打通了如下模块：

1. 运动：能够执行预定义好的动作序列
2. 地标检测：输入相机视频帧，返回检测到的地标坐标和置信度
3. 车道线检测：输入相机视频帧，返回检测到的车道线直线解析式

整体流程如下：

在一个循环中，运动代码控制小车直线前进，但是由于电机、麦轮等差异，小车无法保持绝对的直线，因此，持续检测车道线，解析其函数解析式，当发现小车偏离方向时，控制回正。同时，持续检测地标，当发现地标和自己的相对位置处于某个阈值下时，执行目的动作序列。

具体分析如下：

1. 初始化：

①加载模型: `self.model = LFModel(...)`

②设置初始姿态: `self.ctrl.execute(SetServo(servo=[90, 65]))`，调整摄像头和舵机的角度到一个预设的位置。

③启动车辆: `self.ctrl.execute(Start())`，给车辆控制器发送启动信号

2. 感知 (Sense) - 图像采集与预处理

输入: 原始摄像头图像帧 (`frame`)，从共享内存中读取最新一帧图像。

对图像进行裁剪 (`frame[self.height // 3:, :]`)。实践验证，把上 1/3 天花板部分图像剪裁掉，明显提高检测准确性

输出: 一张被裁剪过的、只包含路面信息的图像 (`img_cropped`)。

3. 分析 (Think) - 推理车道线

操作:

模型推理: 将图像送入 `LFModel` 神经网络，模型会直接预测出图中所有可能的车道线，并给出每条线的斜率 k 、截距 b 和一个置信度分数。

数据筛选与转换: 只保留置信度最高的 4 条车道线，并将其坐标从模型内部的尺寸转换到我们裁剪图像的实际尺寸上。

输出: 列表，是置信度最高的 4 条车道线（已经转换好坐标）

4. 决策 - `steering_command`

输入: 上一步得到的可靠车道线列表。

操作: 调用 `_compute_steering_command` 函数，采用了一个简单的比例控制算法 (P-Control):

计算出左右两条车道线在图像底部构成的车道中心点。

计算这个车道中心点与图像正中心之间的横向像素误差。

将这个“误差”乘以一个比例系数 K_p 。

输出: 一个抽象的转向控制量 `steering_command`

这个值的正负代表转向方向，大小代表转向幅度。

5. 行动 (Act) - 执行最终动作

输入: 上一步计算出的 `steering_command`。

操作: 设定阈值，判断 `steering_command` 的大小，从而决定是行 `TurnLeft`, `TurnRight` 还是 `Advance`；上下界的具体值，可将车调整不同方向，查看终端输出的 `steering_command`

输出: 一个具体的行驶指令，在 `base_action.py` 中已定义虚拟速度和电机编码器关系

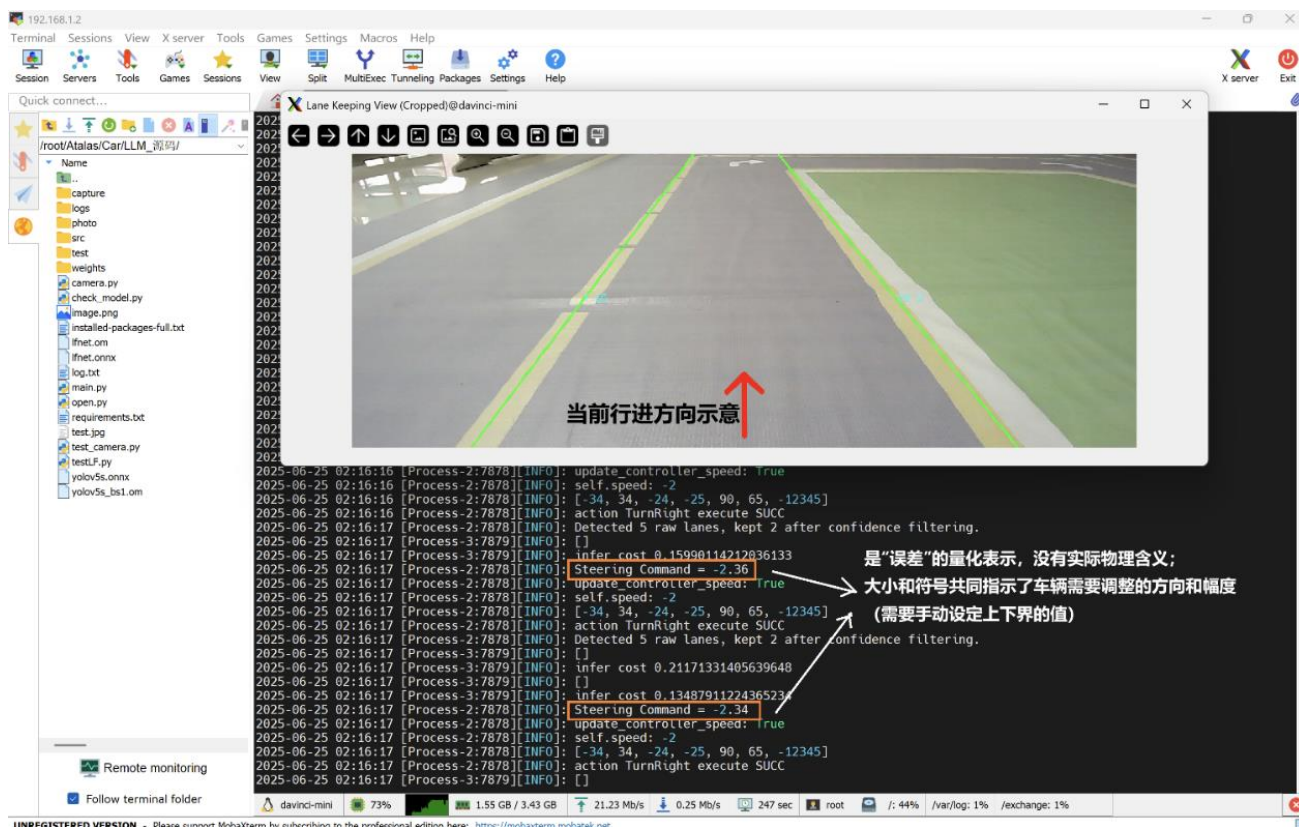


图 9.6.2-12 最终 demo 运行成果

9.4.5 改进

在走完上述流程后，同学们可以思考一些改进策略，例如：

1. 将 TT 电机改完带编码的 TT 电机，使用 pid 内外环对小车行进、转向进行控制。
2. 搭建或者复现一些端到端的车道线矫正模型，输入图像，输出转向角度。
3. 综合现有分散数据集、在数据清洗、统一标注和补充图集的基础上，训练能够适应不同地图的通用模型。
4. 扩展一些新功能，例如语音模块、跟随模块等。
5. 使用 ROS2 系统作为通信机制，让运动、地标、车道线模块之间进行通信
6. 基于大语言模型的 Agent 机制，使用 Agent 作为决策中枢，配合 ROS2 实现新的小车软件架构。

9.5 本章总结

本章阐述了基于 Atlas200I DK A2 昇腾开发板与 ESP32 单片机的智能小车视觉感知与运动控制系统，围绕目标检测、车道线检测、模型训练、嵌入式部署、PID 运动控制五大核心内容展开设计与实现，最终完成车道循迹、行人检测、人行道识别、路口转弯、原地掉头等一系列自主行驶功能。

系统采用上下位机分布式架构，上位机 Atlas200I DK A2 依托 NPU 硬件加速能力，运行经过训练与格式转换的深度学习模型，完成图像采集、视觉推理与高层决策；下位机 ESP32 结合编码电机实现双 PID 闭环控制，接收上位机指令完成直行、转向、停止等运动执行，形成“视觉感知 - 智能决策 - 运动执行”的完整工作闭环。视觉层面，选用 YOLO 单阶段目标检测算法实现行人、人行道、各类交通标识的实时识别，兼顾精度与嵌入式实时性需求；车道线检测提供 LaneNet+H-Net 实例分割拟合、LFNet 端到端回归两套方案，分别适配复杂全场景与简易固定场景，使用者可根据项目需求自主选用。控制层面，系统采用串级双 PID 架构，以上位机角度 PID 作为外环、底盘速度 PID 作为内环，遵循“先调内环、后调外环”的调试原则，保障小车行驶与转向的稳定性。

整套方案融合深度学习、嵌入式开发、自动控制三大技术，架构清晰、功能完整，能够满足教学实验与小型智能车全场景行驶的基本要求。但在实际测试中发现，系统仍存在复杂环境泛化能力不足、多模型并发推理性能受限、传统 PID 控制动态适应性差、串口通信存在延时等问题，仍有较大优化与迭代空间。

后续可对系统通信与决策架构进行整体升级改造，迁移至 ROS2 分布式消息框架以实现各功能模块解耦、标准化话题交互，解决原有串口通信延时、耦合度高的问题；同时分级引入轻量化本地量化大模型及云端大语言模型 Agent 作为高层全局规划单元，依托大模型统筹车道循迹、行人避让、人行道限速、转弯掉头多重行驶约束，替代大量硬编码判断逻辑，底层保留车道检测、目标识别与串级 PID 精细调速模块形成“大模型高层规划+传统控制底层执行”的分层控制体系，再同步配合模型轻量化量化、多传感融合、自适应 PID 等配套优化，全面提升小车环境适应性、行驶平顺性与功能可拓展性。